

DocProcessor

Transformer la documentation en une carte des fonctionnalités vérifiable pour l'automatisation des tests QA.

Résumé

DocProcessor est un module Go autonome et entièrement découplé qui charge la documentation d'un projet, construit des cartes structurées des fonctionnalités et suit la couverture de vérification. Conçu pour fonctionner avec les agents LLM pour une extraction intelligente des fonctionnalités, il intègre également une extraction heuristique pour une utilisation hors ligne.

Description courte

Un module Go agnostique aux projets, dédié au traitement de la documentation et à l'extraction de cartes des fonctionnalités. Il analyse les documents pour en tirer des cartes structurées et suit les fonctionnalités vérifiées — en s'appuyant sur les agents LLM pour une extraction intelligente ou sur des heuristiques en mode hors ligne — afin d'alimenter l'automatisation des tests QA avec une garantie anti-bidon, toujours alignée sur la réalité.

Description détaillée

Toutes les équipes logicielles vivent avec le même mensonge lent : la documentation promet des fonctionnalités, les tests couvrent quelque chose d'approchant, et personne ne peut affirmer en toute confiance si les deux décrivent le même produit. DocProcessor existe pour rendre cet écart visible et mesurable. À partir de la documentation d'un projet, il construit une carte structurée des fonctionnalités — un modèle énuméré et lisible par machine de tout ce que le produit prétend faire — et suit la couverture de vérification par rapport à celle-ci. Ainsi, la question « *cette fonctionnalité documentée est-elle réellement prouvée ?* » cesse d'être un débat de couloir pour devenir une requête avec

une réponse claire. Il fonctionne délibérément en double mode : il utilise les agents LLM pour une extraction intelligente et sémantique des fonctionnalités lorsqu'ils sont disponibles, et bascule sur un analyseur heuristique pour une utilisation entièrement hors ligne. Il ne dépend jamais strictement d'un modèle et s'exécute de manière identique, que ce soit dans un job CI en environnement isolé ou sur l'ordinateur portable d'un développeur en mode avion.

Sur le plan architectural, il s'agit d'un module Go autonome, ignorant les spécificités des projets et entièrement découplé (CONST-051(B)) : il ne contient aucune valeur propre à un projet et est intégré par les utilisateurs comme un sous-module de codebase équivalent, permettant à n'importe quel projet de l'adopter sans hériter des hypothèses d'un autre. Il se soumet lui-même aux standards qu'il impose aux autres : ses propres affirmations sont soumises au pacte anti-bidon (CONST-035) et aux règles de couverture d'automatisation complète (CONST-048), ce qui signifie que chaque capacité annoncée dans son README est testée par un script automatisé ou un *Challenge* confirmant un comportement réel et utilisable par l'utilisateur final, et non simplement une sortie sans erreur. Les chaînes destinées à l'utilisateur passent par le traducteur i18n CONST-046. L'objectif de tout cela est de former une boucle fermée : DocProcessor constitue le point d'entrée du cycle QA que HelixQA referme — il extrait la carte des fonctionnalités à partir de la documentation, HelixQA prouve chaque fonctionnalité cartographiée par des preuves d'exécution capturées, et documentation, tests et comportement livré sont contraints de converger au lieu de s'éloigner discrètement version après version.

Pourquoi l'avons-nous développé

Documentation et tests divergent : les documents promettent des fonctionnalités que aucun test ne prouve, et le QA ne peut pas facilement déterminer ce que signifie « *complet* ». DocProcessor transforme la documentation en une carte des fonctionnalités lisible par machine, permettant ainsi de mesurer la couverture de vérification par rapport à ce qui a réellement été promis.

Contenu

Pourquoi c'est un tournant

Il transforme la question la plus floue de la livraison logicielle — « *ce que nous avons livré correspond-il à ce que nous avons annoncé ?* » — en quelque chose d'automatisable et de vérifiable en continu, et ce sans dépendance rigide à AI : extraction via LLM lorsqu'un modèle est disponible, recours à des heuristiques en son

absence, de sorte que la même garantie s'applique dans tous les environnements, qu'il s'agisse d'un exécutable hors ligne ou d'un pipeline entièrement agentique.

Ce qui est innovant

- Extraction de la cartographie documentation-fonctionnalité avec suivi de la couverture de vérification.
- Extraction double : pilotée par agent LLM ou heuristique/hors ligne.
- Découplage agnostique au projet, sans configuration (CONST-051(B)).
- Auto-vérification anti-bidon : les affirmations du README sont étayées par des tests/Challenges (CONST-035/048).

Défis et solutions

- **Fonctionnement optionnel du modèle** : résolu par un extracteur heuristique en repli, permettant au module de fonctionner hors ligne.
- **Alignement entre documentation et réalité** : résolu par des cartes de fonctionnalités structurées et un suivi de la couverture de vérification, intégrés à la boucle d'assurance qualité.
- **Réutilisabilité** : résolue via un découplage strict et une consommation de sous-modules dans un codebase unifié.
- **Crédibilité de ses propres affirmations** : résolue par des tests/Challenges anti-bidon pour chaque capacité annoncée.

Pile technologique (pourquoi + comment)

- **Go (1.25+)** — cœur du module ; sous licence Apache-2.0.
- **Agents LLM** — extraction sémantique intelligente des fonctionnalités (optionnelle).
- **Analyseur heuristique** — repli pour l'extraction hors ligne des fonctionnalités.
- **Traducteur i18n (pkg/i18n)** — chaînes localisées CONST-046.
- **Harnais de Challenges** — vérification anti-bidon des affirmations du module.